



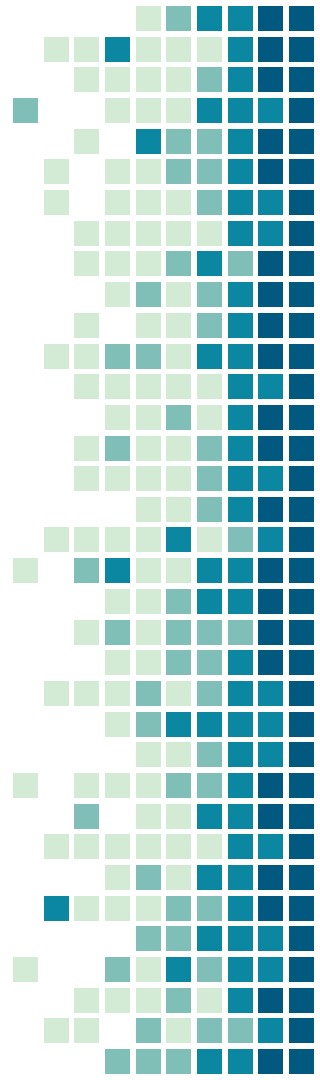
CdL in Informatica – A.A. 2024 – 2025

Programmazione 1 – Modulo 2

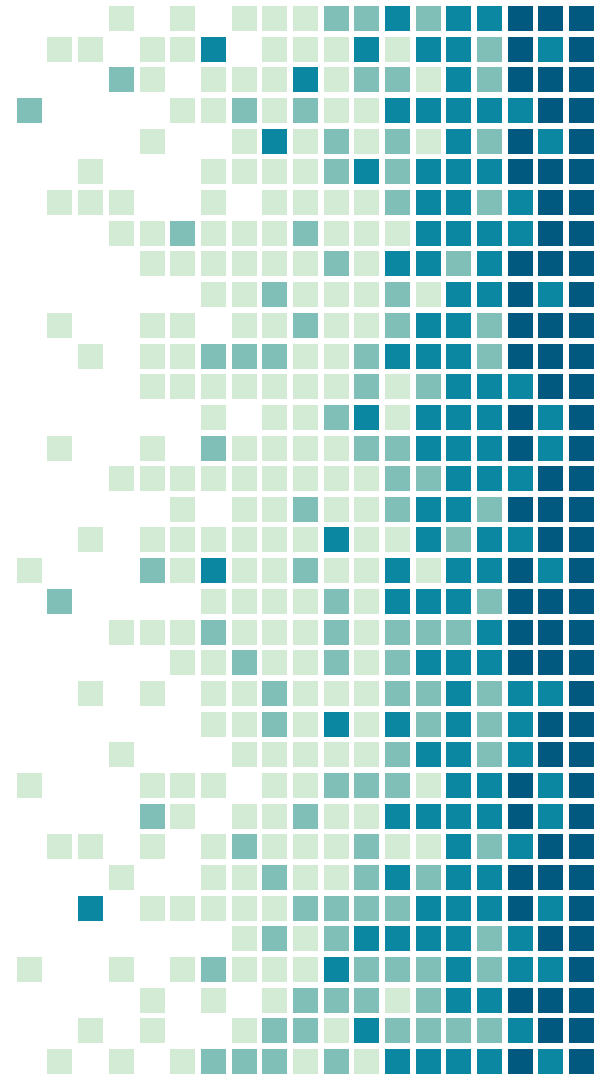
Lezione 11
21/11/2024

Andrea Loddo

Federico Meloni - Alessandra Perniciano - Fabio Pili



I puntatori



La memoria

- **Memoria:** costituita da un insieme di "celle" di dimensione fissata, ognuna con un proprio **indirizzo** sequenziale.
- La **dichiarazione di variabili** indica al compilatore **quante celle** (e quanto grandi) servono.
- Il **tipo di dato** indica **quante celle** sono riservate a una variabile di quel **tipo**
 - GCC: **char**: 1 byte; **int**: 4 byte; **float**: 4 byte; **double**: 8 byte.
- **Dove** vengono memorizzate?
 - I linguaggi ad alto livello **non se ne occupano** e **non ci interessa saperlo**
 - L'OS sa quali indirizzi dare alle variabili sulla base delle risorse disponibili.
- Il linguaggio C permette l'accesso **diretto** alla memoria:
 - Serve una variabile di tipo **puntatore**, che contiene al suo interno un **indirizzo di memoria**
 - Permette di superare alcuni limiti strutturali del linguaggio, ma serve molta **attenzione** nell'utilizzarlo.

...	...							
Indirizzo 4	0	1	1	0	1	0	1	0
Indirizzo 3	1	0	1	0	1	0	0	0
Indirizzo 2	1	0	1	0	1	0	0	1
Indirizzo 1	1	1	0	1	1	0	0	0
Indirizzo 0	0	1	0	1	1	0	0	0

Puntatori: dichiarazione

- **Puntatore:** variabile che contiene un indirizzo di memoria in cui è contenuto un valore di un determinato tipo di dato.
- Sintassi: `tipo *nomeVariabile;`
- Il simbolo `*` indica che la variabile che lo segue è un puntatore al tipo di dato specificato.

```
/* Esempi di dichiarazioni: */  
  
int *puntatore; // puntatore a intero  
  
float *puntatore; // puntatore a float  
  
int *p1, p2; // p1 è un puntatore a intero, p2 è una variabile intera  
  
float *p, *q; // p e q sono puntatori a float  
  
char *puntatore; // puntatore a carattere
```

Puntatori: inizializzazione

- Come per le variabili, è consigliabile inizializzare un puntatore a un valore standard che indichi che tale puntatore non punta a niente:
 - La keyword **NULL** (rigorosamente maiuscolo) è una macro particolare che indica l'indirizzo 0, contenuta nella libreria **stdio**
- L'indirizzo 0 non è utilizzabile da nessun processo
 - Se un puntatore tenta di accedere a quella locazione si verifica un crash del programma (**SEGMENTATION FAULT**).
- Utilità dei NULL pointer: impedire a un processo di utilizzare puntatori **non inizializzati** e accedere a zone di memoria senza controllo.

```
/* Esempio */  
int *puntatore = NULL;  
...  
/* Evita di accedere al puntatore se questo non punta a  
una locazione di memoria valida: */  
if ( puntatore != NULL ) {  
    // esegui operazioni con il puntatore  
}
```

Puntatori: referencia

- Per assegnare un indirizzo a cui far puntare un puntatore è possibile usare l'operatore di referencia **&** (ampersand).
- Anteposto ad una variabile, l'**operatore unario &** restituisce l'indirizzo di memoria della variabile stessa.
- Esempio: se dichiaro una variabile intera **numero** che, al momento dell'esecuzione, viene memorizzata all'indirizzo **0x61fecc**, l'espressione **&numero** viene valutata **0x61fecc**.

```
int numero = 25;  
int *puntatore = NULL;  
puntatore = &numero;
```

Indirizzi	Memoria	
0x61fecc	25	numero
0x61fec8		
0x61fec4		
0x61fec0		
0x61feb8	0x61fecc	puntatore
0x61feb4		
0x61feb0		
0x61feac		
0x61fea8		
0x61fea4		
0x61fea0		

Puntatori: stampa

Per stampare un indirizzo di memoria è possibile usare il segnaposto `%p`.

```
int numero = 25;  
int *puntatore = NULL;  
puntatore = &numero;  
printf("Indirizzo puntato dal "  
      "puntatore: %p", puntatore);
```

```
Indirizzo puntato dal puntatore: 0061FECC
```

Indirizzi	Memoria	
0x61fecc	25	numero
0x61fec8		
0x61fec4		
0x61fec0		
0x61feb8	0x61fecc	puntatore
0x61feb4		
0x61feb0		
0x61feac		
0x61fea8		
0x61fea4		
0x61fea0		

Puntatori: dereferenza

- Si può agire sul contenuto della locazione puntata da un puntatore grazie all'operatore di dereferenza `*` (star).
- L'operatore `*`, anteposto al nome del puntatore, permette di accedere alla locazione di memoria puntata dal puntatore a cui è abbinato.
- Da non confondere con l'asterisco in fase di dichiarazione!

```
int numero = 25, doppio = 0;  
int *puntatore = NULL;  
puntatore = &numero;  
doppio = 2 * (*puntatore);  
printf("Il contenuto di numero e' %d\n", *puntatore);  
printf("Il contenuto di doppio e' %d", doppio);
```

```
Il contenuto di numero e' 25  
Il contenuto di doppio e' 50
```

Indirizzi	Memoria	
0x61fecc	25	numero
0x61fec8	50	doppio
0x61fec4		
0x61fec0		
0x61feb8	0x61fecc	puntatore
0x61feb4		
0x61feb0		
0x61feac		
0x61fea8		
0x61fea4		
0x61fea0		

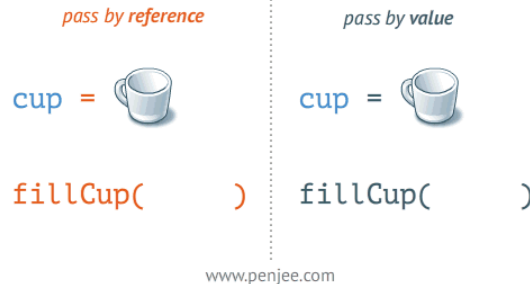
Puntatori: dereferenza

- Si può agire sul contenuto della locazione puntata da un puntatore grazie all'operatore di dereferenza `*` (star).
- Abbiamo visto (slide precedente) come la dereferenza di un puntatore possa essere un *right-value*.
- La dereferenza di un puntatore può anche essere un *left-value*.

```
int numero = 25;  
int *puntatore = NULL;  
puntatore = &numero;  
*puntatore = 100;
```

Indirizzi	Memoria	
0x61fecc	100	numero
0x61fec8		
0x61fec4		
0x61fec0		
0x61feb8		
0x61feb4		puntatore
0x61feb0	0x61fecc	
0x61feac		
0x61fea8		
0x61fea4		
0x61fea0		

REMINDER: Parametri (Lezione 9)



- In generale, esistono due modalità di passaggio dei parametri:
 - **per valore:** i parametri che vengono passati a un sottoprogramma vengono copiati in nuove variabili che sono visibili solo al sottoprogramma.
 - **per indirizzo (o riferimento):** i parametri che vengono passati a un sottoprogramma sono quelli **originali**, NON copiati.
- In linguaggio C, esiste unicamente il passaggio per valore.
 - Gli array, per ora, fanno eccezione a questa regola. Vedremo il motivo quando studieremo i puntatori.
- In particolare, si definiscono i parametri:
 - **attuali:** quelli che il chiamante fornisce al sottoprogramma;
 - **formali:** quelli che il sottoprogramma riceve e utilizza.

REMINDER: subroutine e passaggio di parametri

Passaggio di parametri per **valore**: le subroutine ricevono dei parametri formali che sono **copie** dei parametri attuali passati dal chiamante.

- Quindi: modificare il parametro in una subroutine non causa la modifica della variabile passata al momento della chiamata.
- In altre parole: le modifiche al parametro formale NON si riflettono sul parametro attuale.

```
/* Osservazioni:  
1) il parametro num è una copia del  
   parametro num del main.  
2) num2 è una variabile locale,  
   diversa da num2 del main */  
int doppio( int num )  
{  
    int num2;  
    num2 = 2*num;  
    return num2;  
}  
  
int main()  
{  
    int num, num2;  
    num = 5;  
    num2 = doppio(num);  
}
```

Subroutine e puntatori: esempio 1

```
void doppio( int *p )
{
    (*p) = 2*(*p);
    p = NULL;
}

int main()
{
    int b = 5;
    int *p = &b;
    doppio(p);
}
```

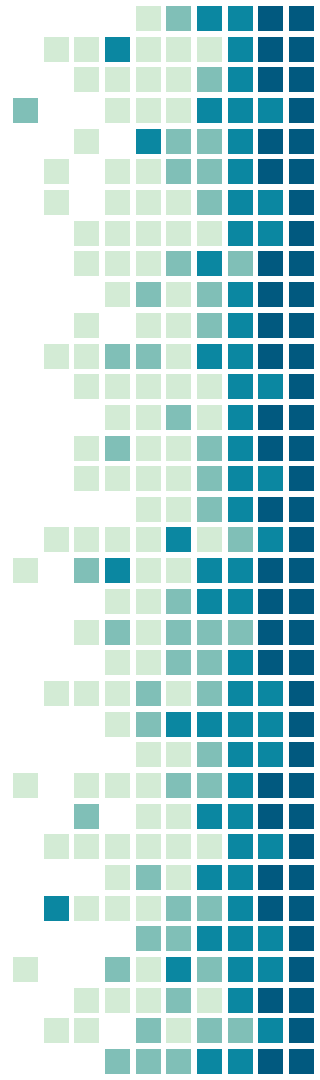
1. La subroutine doppio riceve un puntatore a intero, **p**.
2. Assegna, alla cella di memoria puntata da p, il doppio del valore contenuto nella cella di memoria puntata da p.
3. Dato che il parametro p è un puntatore (contiene un indirizzo), passare p significa dare la **copia** della referenza (l'indirizzo in memoria) della variabile b dichiarata nel *main()*.
4. Di conseguenza, dopo la chiamata a *doppio()*, la variabile b del *main()* risulterà modificata.
5. Il p ricevuto da *doppio()* è una **copia** del p del *main()*, quindi assegnargli NULL dentro *doppio()* non comporterà nessuna variazione nell'indirizzo contenuto nella variabile p del *main()*.

Subroutine e puntatori: esempio 2

```
int subroutine( int x, int y ){  
    int sum = x+y;  
    int dif = x-y;  
    return sum;  
    return dif; // viene ignorata  
}
```

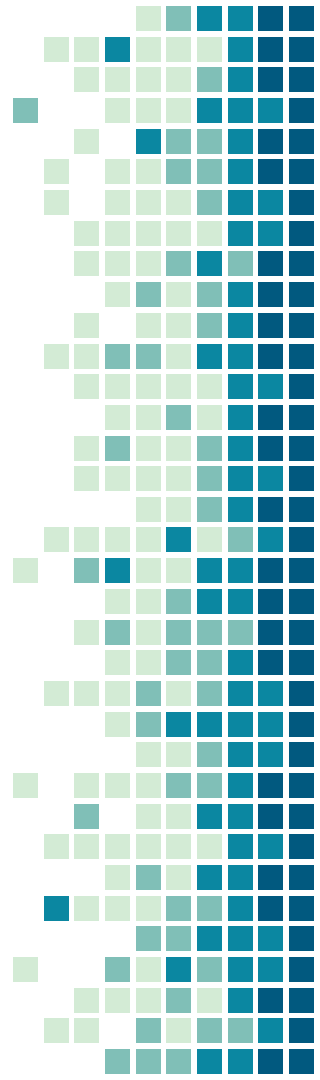
```
void subroutine(int x, int y, int *somma, int *diff)  
{  
    *somma = x+y;  
    *diff = x-y;  
}  
  
int main( )  
{  
    int val1=10, val2=20, somma, diff;  
    subroutine( val1, val2, &somma, &diff );  
  
    printf("Somma: %d\n", somma);  
    printf("Differenza: %d", diff);  
}
```

- Una subroutine che modifica due valori non è implementabile con le return;
- Soluzione: posso passare alla subroutine più puntatori ai valori che intendo modificare.
- Al momento della chiamata, viene passato l'indirizzo di somma e diff.



Passaggio di parametri "per indirizzo"

- Cosa succede se passiamo un puntatore come parametro a una subroutine?
 - La subroutine può usare il puntatore per modificare la variabile.
- Cosa comporta?
 - Si supera il limite del passaggio per valore, perché non si agisce più su una copia del valore, ma **direttamente** sulla cella di memoria in cui il dato è memorizzato.
 - Si **aggirano le regole di scoping**, perché i cambiamenti diventano visibili a tutti gli ambienti che hanno accesso a tale indirizzo.
 - Si evita il limite di un solo valore restituibile per funzione, perché passando più puntatori come parametri, è possibile modificare più valori e, potenzialmente, non serve un return.
- È un vero e proprio passaggio per indirizzo?
 - No. Stiamo solamente passando una **copia** del puntatore.



Aritmetica dei puntatori

- L'aritmetica e le operazioni permesse sui puntatori hanno un significato diverso rispetto all'aritmetica dei tipi numerici: un puntatore deve essere associato a un tipo di dato pre-esistente.

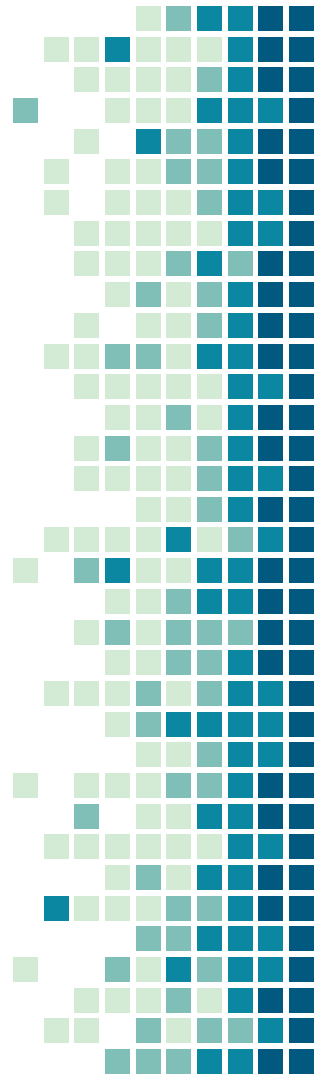
```
int var = 5;
int *p = &var; /* dichiara un puntatore a intero che
                punta alla locazione di memoria in cui è
                memorizzata var */

printf("Il puntatore punta a: %p e contiene %d\n", p, *p);
p++;
printf("Il puntatore punta a: %p e contiene %d", p, *p);
```

- Gli indirizzi in memoria sono **sequenziali**: significa che aggiungendo **n** a un indirizzo indico l'**n-sima** cella di memoria dopo quell'indirizzo.
- Il tipo di dato influenza l'aritmetica degli spostamenti e il modo in cui si accede ai dati in memoria.
- Di quanto aumenta p? L'operazione è "sicura"?

Puntatori e array

- Se non sappiamo cosa c'è in memoria nella cella dopo (può esserci di tutto!) a cosa può servire incrementare (o decrementare) un puntatore?
- L'utilità dell'incremento di un puntatore si concentra nella corrispondenza tra **memoria** e **array**.
- Il nome di un vettore in C (l'identificatore, senza indici e senza parentesi quadre) è un'espressione che viene valutata come l'indirizzo di memoria della sua prima posizione.
- Per come un array viene memorizzato, spostare un puntatore di una posizione equivale a spostare un indice all'interno di un array.
- Uscire dai limiti dell'array (con indici o puntatori) può causare **segmentation fault**: accesso a zone di memoria non consentite.
- NB: non è possibile riassegnare i puntatori associati ai nomi dei vettori perché sono **puntatori costanti**.



Puntatori e array: esempi

```
int v[] = {10,20,30,40,50};
int i;
int *p = NULL;
p = v; //non &v, v è già puntatore

for( i=0; i<5; i++ )
{
    printf("%d ", *p );
    p++;
}
```

```
int v[] = {10,20,30,40,50};
int i;
int *p = NULL;
p = &v[0]; // prima locazione array

for( i=0; i<5; i++ )
{
    printf("%d ", p[i] );
}
```

- Il nome di un vettore (l'identificatore, senza indici e senza parentesi quadre) è un'espressione che viene valutata come l'**indirizzo di memoria della sua prima posizione**.
- Se viene dichiarato un array v e un puntatore p, scrivere **p=v**; equivale a scrivere **p = &v[0]**;
- Inoltre, l'operatore di indicizzazione **[]** può essere associato a un puntatore per accedere alle locazioni successive esattamente come per un array.

Passaggio di array

- In C, gli array fanno "eccezione alla regola del passaggio per valore" perché il nome di un array (senza []) è l'indirizzo dell'elemento di indice 0.
- Il C offre una sintassi per il passaggio di array come parametri in modo che, per chi legge il codice, possa essere evidente che la subroutine lavora con un array:

```
int somma(int dim, int arr[]); // la funzione somma riceve arr[]. È chiaro che sia un array.  
/* scrittura equivalente, ma chi legge il codice non sa che arr è un array direttamente dal  
   prototipo. Finché non si legge la definizione, potrebbe essere un puntatore semplice. */  
int somma(int dim, int *arr);
```

```
/* Esempio: è chiaro dal prototipo che la subroutine lavorerà con un array. Avrei  
   potuto passare int *a, ma non sarebbe stato esplicito per il lettore che si  
   tratta di un array. */  
void incrementaArray( int dim, int a[] ){  
    for(int i=0; i<dim; ++i)  
        a[i]++;  
}  
int main(){  
    int v[] = {1, 2};  
    incrementaArray(2, v);  
}
```

Passaggio di strutture

```
typedef struct
{
    float peso;
    int altezzaCm;
    char cognome[20];
} Calciatore;
```

- Un puntatore a struttura si dichiara allo stesso modo degli altri puntatori:

```
Calciatore c = {95.0, 195, "Pavoletti"}; // Peso, altezza, cognome
Calciatore *cp = &c; // puntatore a variabile struttura Calciatore
```

- La differenza tra **struttura** e **puntatore a struttura** risiede nel modo in cui si accede ai suoi campi:
 - Si può accedere ai campi dereferenziando il puntatore e utilizzando l'operatore **punto**:

```
printf( "%s", (*cp).cognome );
```

- Oppure utilizzando l'operatore freccia -> (trattino e simbolo maggiore) applicato direttamente sul puntatore:

```
printf( "%s", cp->cognome );
```

NB: l'operatore freccia esiste per evitare le parentesi tonde nella dereferenziazione che sarebbero necessarie per via delle regole di **precedenza degli operatori**.

FINE!

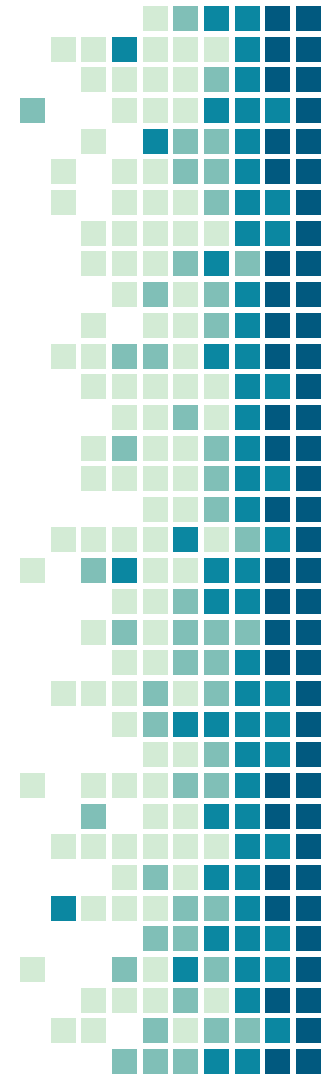
Domande?

Autovalutazione

1. L'accesso alla memoria è un'operazione priva di rischi?
2. È possibile effettuare operazioni matematiche sui puntatori?
3. Cosa contiene una variabile puntatore?
4. È possibile creare puntatori a tipi di dato definiti dal programmatore?
5. Sono in grado di definire un puntatore a intero, float, char o a un tipo definito da me?
6. Cosa definisce la sintassi `int * a, b, *c; ?`
7. Qual è l'operatore di referenza?
8. A cosa serve il segnaposto `"%p"` ?
9. È possibile modificare il contenuto di una locazione di memoria tramite il suo puntatore?
10. Qual è l'operatore di dereferenza?
11. A quale valore andrebbe inizializzato un puntatore?
12. Cosa rappresenta la keyword `NULL` e per cosa si usa?
13. Per cosa può essere utile passare come parametri gli indirizzi di memoria e non i valori stessi?
14. Come si può accedere a un array passato a una funzione mediante puntatore?
15. Come si può accedere a una variabile di tipo struttura passata a una funzione mediante puntatore?

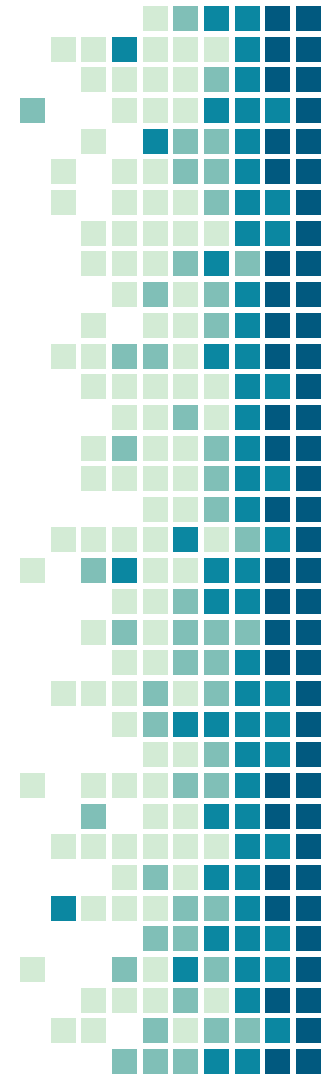
Esercizi

1. Scrivere un programma per lo scambio di due variabili. In particolare, le due variabili vanno dichiarate nel main e lo scambio deve essere effettuato da una procedura. Stampare il valore delle variabili prima e dopo lo scambio.
2. Scrivere una funzione che controlli se due vettori sono uguali e restituisca true in caso affermativo, false altrimenti.
3. Scrivere un programma che invochi una procedura che, ricevuti in ingresso tre valori e l'indirizzo di memoria di due variabili (min e max) definite nel main, memorizzi i valori minimo e massimo all'interno delle relative variabili. Effettuare la stampa poi direttamente nel main.
4. Scrivere una subroutine che stampi un array ricevuto come parametro.
5. Scrivere una subroutine che stampi una matrice, utilizzando la subroutine scritta per l'esercizio 4.
6. Scrivere un programma che, con l'ausilio di una procedura, trovi il massimo e minimo di un array di N elementi. In particolare, l'array, il massimo e il minimo sono dichiarati nel main e l'array deve essere inizializzato casualmente mediante una procedura. Stampare vettore, massimo e minimo nel main.
7. Utilizzare la subroutine dell'esercizio precedente per trovare minimo e massimo presenti su ogni riga di una matrice.



Esercizi

8. Scrivere una funzione di ricerca di un elemento di un array. Se l'elemento passato come parametro si trova nell'array la funzione ne restituisce l'indice di posizione, altrimenti restituisce -1.
9. Scrivere una procedura che stampi il contenuto di un vettore senza usare indici.
10. Definire una struttura 'Casa' composta dai campi 'numeroPiani', 'mq', 'prezzo' che permetta di acquisire i relativi dati attraverso una procedura passando come parametro l'indirizzo di memoria della stessa. Scrivere poi una seconda procedura che ne permetta la stampa.
11. Scrivere una funzione che verifichi se c'è una sovrapposizione tra due intervalli (min1,max1) e (min2,max2). Se sì, la funzione restituirà vero e il punto medio della loro sovrapposizione si troverà in un argomento di output, altrimenti renderà falso.
12. Scrivere un programma in cui è dichiarato un vettore v, di dimensione N. Il vettore deve essere popolato con numeri interi, positivi e negativi, generati casualmente. Scrivere poi una funzione che riceve come parametro il vettore e inverte l'ordine degli elementi. Visualizzare dal main sia il vettore originale che quello ottenuto dopo la chiamata della funzione.
13. Scrivere una funzione che restituisca true se la stringa passata come parametro è palindroma, false altrimenti.



Esercizi

14. Definire una struttura per la modellazione dei dati anagrafici di uno studente (nome, cognome, data di nascita etc), quindi creare e popolare un array di studenti. Aggiungere una seconda struttura per la modellazione dei risultati di una sessione d'esame che contiene la data, il luogo, il nominativo del professore, e un elenco di voti abbinati agli studenti (suggerimento: utilizzare gli indici degli studenti nel primo array per recuperare le informazioni).

Realizzare:

- una funzione che permetta di acquisire i dati di interesse di uno studente;
- una procedura che, utilizzando la funzione precedente, popoli un array di studenti dichiarato nel main;
- una procedura che permetta di acquisire i dati di un esame sostenuto dagli studenti;
- una funzione che verifica che lo stesso studente non compaia due volte nell'elenco;
- una funzione che conta quanti studenti hanno sostenuto l'esame con esito positivo;
- una funzione che visualizza i nomi e i voti degli studenti che hanno ottenuto voto maggiore o uguale a 28.

Testare le varie funzioni e procedure dal main.